МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

з дисципліни "Функціональне програмування"

НА ТЕМУ "ЛІНИВІ ОБЧИСЛЕННЯ: ГЕНЕРАТОРИ ТА ITERTOOLS"

Варіант 12. Потік трикутних чисел

Виконав:
студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів — 2026

Мета: засвоїти концепцію лінивих обчислень у Python, опанувати роботу з нескінченними послідовностями за допомогою функцій-генераторів із `yield`/`yield from`, вивчити базові будівельні блоки модуля `itertools` (`count`, `islice`, `accumulate`, `takewhile`, `dropwhile`), реалізувати базовий функціонал (нескінченний стрім Фібоначчі, `sliding_window`, `take`, `drop`, `moving_average`) та індивідуальне завдання варіанта 12 (лінивий потік трикутних чисел, фільтрацію до заданої межі та обчислення різниць сусідніх значень за допомогою ковзних вікон).

Теоретичні відомості

У Python розрізняють *ітеровані об'єкти* (iterable) та *ітератори* (iterator). Ітератор — це об'єкт зі станом, який за викликом методу `__next__()` повертає наступне значення або генерує виняток `StopIteration` після вичерпання. Концепція *лінивих обчислень* (lazy evaluation) полягає в тому, що елементи послідовності обчислюються виключно в момент запиту споживачем. Це дозволяє ефективно оперувати потенційно нескінченними чи надвеликими потоками даних із просторовою складністю *O(1)* пам'яті.

Основними інструментами побудови лінивих конвеєрів є генераторні вирази, функції з ключовим словом `yield`/`yield from` та набір комбінаторів модуля `itertools`. Генератор автоматично запам'ятовує локальний стан і кадр виконання між викликами. Для коректної роботи з нескінченними джерелами споживання має строго обмежуватися за допомогою `islice`, `takewhile` або власних утиліт `take`/`drop`, уникаючи передчасної матеріалізації в структури `list` або `tuple`.

Хід роботи

1. Базові лініві генератори та утиліти (streams.py): реалізовано генератор нескінченного потоку чисел Фібоначчі `fib_stream`, нескінченний лічильник натуральних чисел `naturals`, утиліти обмеження та пропуску `take` і `drop` з повною валідацією аргументів (обробка `n < 0`), а також генератор ковзного вікна `sliding_window` через фіксований буфер `deque(maxlen=k)` і `moving_average`:

```
from collections import deque
from itertools import count, islice
from typing import Iterable, Iterator, TypeVar

T = TypeVar("T")

def fib_stream() -> Iterator[int]:
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

def naturals(start: int = 1) -> Iterator[int]:
    yield from count(start)

def take(n: int, it: Iterable[T]) -> list[T]:
    if n < 0:
        raise ValueError("n must be non-negative")
    return list(islice(it, n))
```

```
def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    if n < 0:
        raise ValueError("n must be non-negative")
    iterator = iter(it)
    next(islice(iterator, n, n), None)
    return iterator

def sliding_window(it: Iterable[T], k: int) -> Iterator[tuple[T, ...]]:
    if k <= 0:
        raise ValueError("k must be strictly positive")
    iterator = iter(it)
    dq: deque[T] = deque(maxlen=k)
    for _ in range(k):
        try:
            dq.append(next(iterator))
        except StopIteration:
            return
    yield tuple(dq)
    for x in iterator:
        dq.append(x)
        yield tuple(dq)

def moving_average(it: Iterable[float | int], k: int) -> Iterator[float]:
    for win in sliding_window(it, k):
        yield sum(win) / k
```

2. Реалізація завдань Варіанта 12 (triangular_numbers.py): згідно з варіантом 12 розроблено лінивий потік трикутних чисел `triangular_stream()`, де $T_n = 1 + 2 + ... + n$ формується без повторних обчислень за допомогою `itertools.count(1)` та `itertools.accumulate`. Також реалізовано фільтрацію за порогом `triangular_up_to(limit)` через `takewhile` та обчислення різниць сусідніх значень `triangular_differences(it)` через `sliding_window(it, 2)`, що відновлює вихідний крок прогресії:

```
from itertools import accumulate, count, takewhile
from typing import Iterable, Iterator
from streams import sliding_window

def triangular_stream() -> Iterator[int]:
    yield from accumulate(count(1))

def triangular_stream_gen() -> Iterator[int]:
    total = 0
    for n in count(1):
        total += n
        yield total

def triangular_up_to(limit: int) -> Iterator[int]:
    return takewhile(lambda x: x <= limit, triangular_stream())

def triangular_differences(it: Iterable[int]) -> Iterator[int]:
    for prev, curr in sliding_window(it, 2):
        yield curr - prev
```

3. Модульне та Property-based тестування: створено набір pytest-тестів, що включає перевірку лінивості (лічильник операцій підтверджує, що обчислюються лише запитані елементи), граничні випадки (порожні послідовності, `k > len`, некоректні `n < 0`, `k <= 0`), а також property-тести Hypothesis:

```
def test_laziness_evaluation() -> None:
    eval_count = 0
    def side_effect_stream():
        nonlocal eval_count
        for i in count(1):
            eval_count += 1
            yield i

    stream = side_effect_stream()
    result = take(5, stream)
    assert result == [1, 2, 3, 4, 5]
    assert eval_count == 5

def test_triangular_differences_property() -> None:
    triangles = take(10, triangular_stream())
    diffs = list(triangular_differences(triangles))
    assert diffs == [2, 3, 4, 5, 6, 7, 8, 9, 10]
```

4. Демонстраційний скрипт main.py: створено точку входу для запуску демонстрації конвеєрів, перевірки лінивості та вимірювання часу обчислень:

```
from __future__ import annotations
from streams import drop, fib_stream, naturals, sliding_window, take
from triangular_numbers import triangular_differences, triangular_stream,
triangular_up_to

def main() -> None:
    print("=== Лабораторна робота 5: Лініві обчислення ===")
    print("1. Перші 10 чисел Фібоначчі:", take(10, fib_stream()))
    print("2. Пропуск 5, взяття 8 Фібоначчі:", take(8, drop(5, fib_stream())))

    win_demo = list(sliding_window([1, 2, 3, 4, 5], 3))
    print("3. Sliding window k=3 на [1,2,3,4,5]:", win_demo)

    triangles_10 = take(10, triangular_stream())
    print("4. Варіант 12. Перші 10 трикутних чисел:", triangles_10)

    triangles_limit = list(triangular_up_to(40))
    print("5. Варіант 12. Трикутні числа <= 40:", triangles_limit)

    diffs = list(triangular_differences(triangles_limit))
    print("6. Варіант 12. Різниці сусідніх значень:", diffs)

if __name__ == "__main__":
    main()
```

5. Результати виконання демонстраційної програми (main.py):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab05> python main.py
=== Лабораторна робота 5: Лініві обчислення ===
1. Перші 10 чисел Фібоначчі: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
2. Пропуск 5, взяття 8 Фібоначчі: [5, 8, 13, 21, 34, 55, 89, 144]
3. Sliding window k=3 на [1,2,3,4,5]: [(1, 2, 3), (2, 3, 4), (3, 4, 5)]
4. Варіант 12. Перші 10 трикутних чисел: [1, 3, 6, 10, 15, 21, 28, 36, 45, 55]
5. Варіант 12. Трикутні числа <= 40: [1, 3, 6, 10, 15, 21, 28, 36]
6. Варіант 12. Різниці сусідніх значень: [2, 3, 4, 5, 6, 7, 8]
```

6. Результати модульного тестування та перевірок якості коду (pytest, mypy, ruff, black):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab05> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.3, pytest-8.3.2, pluggy-1.5.0
```

```
plugins: hypothesis-6.115.0
collected 25 items

tests/test_accumulate.py::test_accumulate_basic PASSED                   [  4%]
tests/test_accumulate.py::test_accumulate_mul PASSED                     [  8%]
tests/test_fib_stream.py::test_first_10_fib PASSED                       [ 12%]
tests/test_fib_stream.py::test_fib_drop_and_take PASSED                  [ 16%]
tests/test_sliding_window.py::test_sliding_window_basic PASSED           [ 20%]
tests/test_sliding_window.py::test_sliding_window_k_greater_than_len PASSED [ 24%]
tests/test_sliding_window.py::test_sliding_window_invalid_k PASSED        [ 28%]
tests/test_sliding_window.py::test_sliding_window_empty PASSED           [ 32%]
tests/test_sliding_window.py::test_moving_average PASSED                 [ 36%]
tests/test_take_drop.py::test_take_basic PASSED                          [ 40%]
tests/test_take_drop.py::test_drop_basic PASSED                          [ 44%]
tests/test_take_drop.py::test_take_drop_on_count PASSED                  [ 48%]
tests/test_take_drop.py::test_negative_n_raises PASSED                   [ 52%]
tests/test_take_drop.py::test_laziness_evaluation PASSED                 [ 56%]
tests/test_take_drop.py::test_property_take_drop_count PASSED            [ 60%]
tests/test_triangular.py::test_triangular_stream_values PASSED           [ 64%]
tests/test_triangular.py::test_triangular_generators_parity PASSED       [ 68%]
tests/test_triangular.py::test_triangular_up_to PASSED                   [ 72%]
tests/test_triangular.py::test_triangular_differences PASSED             [ 76%]
tests/test_triangular.py::test_triangular_differences_empty_and_single PASSED [ 80%]
tests/test_triangular.py::test_triangular_laziness PASSED                [ 84%]
tests/test_triangular.py::test_triangular_differences_property PASSED    [ 88%]
tests/test_triangular.py::test_triangular_closed_form_property PASSED    [ 92%]
tests/test_benchmark.py::test_benchmark_streams PASSED                   [ 96%]
tests/test_benchmark.py::test_benchmark_triangular PASSED               [100%]

============================== 25 passed in 1.08s ==============================

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab05> python -m mypy .
Success: no issues found in 6 source files

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab05> python -m ruff check .
All checks passed!

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab05> python -m black --check .
All done! 6 files would be left unchanged.
```

Висновок

Під час виконання лабораторної роботи було всебічно вивчено механізми лінивих обчислень у середовищі Python. Реалізовано універсальні функції-генератори для нескінченних джерел (послідовність Фібоначчі, трикутні числа), утиліти конвеєрної обробки `take`, `drop`, ковзні вікна `sliding_window` через фіксовану двобічну чергу `deque`, а також префіксні згортки `accumulate` та фільтри `takewhile`. Перевірено відсутність надлишкової матеріалізації даних у пам'яті за допомогою тесту лінивості, захищено функції від граничних та невалідних значень аргументів. Проєкт успішно протестовано за допомогою бібліотек pytest і Hypothesis та перевірено інструментами статичного аналізу mypy, ruff і black.